

PRD: OmniMail AI (Universal Email PWA)

1. Vision & Strategy

OmniMail AI is a mobile-first, AI-native email client designed to eliminate "inbox anxiety." It focuses exclusively on the email protocol, using a multi-agent architecture to handle synchronization, synthesis (summarization), and smart composition.

Core Philosophy: The user should never have to "read" an email to understand its intent or priority.

2. Product Features

A. Core Email Operations

Feature	Description
Universal Auth	OAuth2 flow for Gmail/Office 365; IMAP/SMTP credentials for Yahoo/AOL.
Unified Inbox	Aggregate view of all accounts with clear "origin" markers.
Thread Management	Proper nesting of replies, labels/folders mapping, and archive/delete actions.
Rich Compose	Markdown support, attachments, and multi-account "From" switching.

B. AI-First Intelligence

- The Pulse (Executive Summary):** At the top of the inbox, a generated summary of the last 24 hours of unread mail.

- **Smart Drafts:** One-tap reply generation based on the thread context and user "voice" (Professional, Casual, Short).
 - **Saliency Scoring:** Instead of just "Important," AI assigns a priority score (0-10) based on urgency and sender relationship.
-

3. Technical Architecture (Agent OS Approach)

To align with the **Agent OS methodology**, the system is divided into **Agents** (logic controllers) and **Skills** (reusable capabilities).

A. The Agent Registry

1. **Sync Agent:** Manages the delta-sync between the client and IMAP/OAuth providers.
2. **Scribe Agent:** Responsible for generating summaries and reply drafts.
3. **Triage Agent:** Analyzes incoming headers/body to assign labels and priority scores.
4. **Orchestrator:** The primary entry point that routes user intent (e.g., "Find the flight info from last week") to the specific agent.

B. Skills/Hooks Directory

- `useEmailProtocol`: Hook for abstraction over Gmail API vs. IMAP.
 - `skill-vector-search`: Local search capability using embeddings (optional/advanced).
 - `skill-summarizer`: Direct interface with LLM for context-window management.
-

4. UI/UX Requirements

- **Mobile-First PWA:** `manifest.json`, service workers for offline viewing, and "Add to Home Screen" optimization.
 - **The "Swipe" Language:** Left-swipe to archive, long-right-swipe to trigger AI summary.
 - **Zero-Latency Feel:** Optimistic UI updates for all email actions (Archive/Delete should happen instantly in UI while syncing in background).
-

5. Implementation Roadmap for Claude Code

Phase 1: Foundation (Specs-Driven)

- Initialize project using Next.js (App Router) + Tailwind + Shadcn/UI.
- Create `CLAUDE.md` to define project rules, build commands, and test patterns.
- Define `specs/protocol.md` for the unified email schema.

Phase 2: Protocol Layer (Skills)

- Implement OAuth2 for Google/Microsoft.
- Build the IMAP bridge (using a library like imapflow or a backend proxy).
- Test: Ensure a "Unified View" can fetch 10 emails from two different sources.

Phase 3: AI Layer (Agents)

- Integrate Vercel AI SDK.
- Implement the **Triage Agent** hook that runs on every new message fetch.
- Build the **Scribe Agent** for the "Smart Reply" drawer.

Phase 4: Delivery

- PWA configuration (icons, theme colors, offline cache).
- Deploy to Vercel.
- Automated E2E tests (Playwright) for the login-to-inbox flow.

6. CLAUDE.md Template (Standardized)

Copy this into your root directory before starting Claude Code.

Markdown

OmniMail AI Project Guidelines

Build Commands

- Dev: ``npm run dev``
- Build: ``npm run build``
- Test: ``npm run test``

Architecture: Agent OS

- ****Agents**** live in ``src/agents/``: Logic controllers that make decisions.
- ****Skills**** live in ``src/skills/``: Stateless utility functions/hooks.
- ****Hooks**** live in ``src/hooks/``: React-specific lifecycle logic.

Code Style

- Use TypeScript for all files.
- Functional components with Tailwind CSS.
- Use ``lucide-react`` for icons.
- All API calls must be wrapped in error boundaries.

Testing Strategy

- Unit tests for Agents logic in `src/\_\_tests\_\_/agents`.
- E2E tests for the Unified Inbox flow in `tests/e2e`.

7. Success Metrics for Assignment

- **Zero-Config Start:** Claude Code can run npm test and see a passing "Protocol Mock" test immediately.
- **Context Awareness:** AI replies actually reference the specific contents of the previous email.
- **Mobile Fidelity:** PWA score on Lighthouse > 90.

Next Step: Initialize your repository, create the CLAUDE.md file above, and run:

```
claude "Initialize a Next.js PWA based on the specs in PRD.md and set up the Agent OS directory structure."
```

What specific email provider (Gmail, Outlook, or generic IMAP) would you like to prioritize for the initial authentication setup?